Math Foundations Cheat Sheet

Linear Algebra · Calculus · Probability · Matrix Calculus

LINEAR ALGEBRA — MUST KNOW

MATRIX MULTIPLY

$(AB)_{ij} = \Sigma_k A_{ik}B_{kj}$

dims: $(m\times n)(n\times p)\to(m\times p)$

DOT PRODUCT

$a\cdot b = \Sigma_i a_i b_i = \|a\|\|b\|\cos(\theta)$

EIGENVECTORS

$Av = \lambda v$

v = eigenvec, λ = eigenval

TRANSPOSE RULES

$(AB)^T = B^TA^T$

$(A+B)^T = A^T+B^T$

PROBABILITY — LLM-RELEVANT CONCEPTS

BAYES THEOREM

$P(A|B) = P(B|A)P(A)/P(B)$

KL DIVERGENCE

$KL(P\|Q) = \Sigma P(x)\log(P(x)/Q(x))$

≥ 0, not symmetric

CROSS-ENTROPY LOSS

$H(y,\hat{y}) = -\Sigma y_i\log(\hat{y}_i)$

= NLL for LM training

MLE OBJECTIVE

$\theta^* = \text{argmax}\ \Sigma \log P(x_i|\theta)$

= argmin NLL

CALCULUS — CHAIN RULE + GRADIENTS

CHAIN RULE (FUNDAMENTAL FOR BACKPROP)

$dz/dx = (dz/dy)(dy/dx)$

$\partial L/\partial x = (\partial L/\partial y)(\partial y/\partial x)$

GRADIENT OF COMMON FUNCTIONS

$\partial(Wx)/\partial W = x^T$ $\partial(Wx)/\partial x = W^T$

$\partial(\|x\|^2)/\partial x = 2x$ $\partial(a^Tx)/\partial x = a$

SOFTMAX GRADIENT (DERIVE IN INTERVIEWS)

$\partial\sigma_i/\partial z_j = \sigma_i(\delta_{ij} - \sigma_j)$

$\delta_{ij} = 1$ if $i=j$, else 0

When $i=j$: $\sigma_i(1-\sigma_i)$. When $i\neq j$: $-\sigma_i\sigma_j$. The Jacobian of softmax is a full matrix, not a diagonal.

BACKPROP ON 2-LAYER MLP — STEP BY STEP

- **F** **Forward:** $z_1=W_1x+b_1$, $a_1=\text{ReLU}(z_1)$, $z_2=W_2a_1+b_2$, $\hat{y}=\text{softmax}(z_2)$
- **L** **Loss:** $L = -\Sigma y_i\log(\hat{y}_i)$ (cross-entropy)
- **1** $\partial L/\partial z_2 = \hat{y} - y$ (softmax + CE gradient, clean result)
- **2** $\partial L/\partial W_2 = (\partial L/\partial z_2)\cdot a_1^T$ $\partial L/\partial b_2 = \partial L/\partial z_2$
- **3** $\partial L/\partial a_1 = W_2^T\cdot(\partial L/\partial z_2)$
- **4** $\partial L/\partial z_1 = (\partial L/\partial a_1)\odot \text{ReLU}'(z_1)$ [$\odot$ = elementwise, ReLU'=1 if z>0]
- **5** $\partial L/\partial W_1 = (\partial L/\partial z_1)\cdot x^T$ $\partial L/\partial b_1 = \partial L/\partial z_1$

⚑ Numerical gradient check: $\partial L/\partial w \approx [L(w+\varepsilon) - L(w-\varepsilon)] / 2\varepsilon$. Use ε=1e−5. Analytical and numerical gradients should match to 1e−5 precision.

ALL MONTHS · TIMELINE

38-Week Roadmap — Full Timeline

Anthropic · OpenAI · DeepMind · ~820 hours total

W0
Math Prerequisites
Linear algebra, calculus, prob

W1 · M1
Transformer Architecture
Attention, RoPE, ALiBi

W2 · M1
BERT + Scaling Laws
Kaplan, Chinchilla, BPE

W3 · M1 ★
Modern Architectures
MoE, Mamba/SSM, GQA

W4 · M1
RAG Evaluation
RAGAS, BM25, hybrid search

W5 · M2
RAG + Security
HyDE, reranking, Prompt Guard

W6 · M2
Production RAG API
FastAPI, Redis, Docker, CI

W7 · M2
LoRA + QLoRA + DoRA
Fine-tuning foundations

W8 · M2
DPO + SimPO
Alignment, preference data

W9 · M3
Data Curation
Synthetic data, Self-Instruct

W10 · M3
vLLM Serving
PagedAttention, benchmarks

W11 · M3
RAFT
RAG + fine-tuning combined

W12 · M3 ✦
Capstone #2
Fine-tuned LLM pipeline

W13 · M4
ReAct + CoT + ToT
Agent foundations

W14 · M4
Structured Outputs
Instructor, tool use, AgentBench

W15 · M4
Multi-Agent + STORM
Supervisor/worker pattern

W16 · M4 ✦
Capstone #3 + MCP
Research agent + MCP server

W17 · M5
CI Evals
DeepEval, pytest, GitHub Actions

W18 · M5 ★
LLM Security
GCG, red-teaming, Garak

W19 · M5
Kubernetes
k8s, HPA, Prometheus, Grafana

W20 · M5 ✦
Capstone #1
Production RAG API

W21 · M6
Distributed Training
FlashAttention, DDP, ZeRO

W22 · M6
Mech. Interpretability
SAE, superposition, steering

W23 · M6
RLHF + CAI + RLAIF
InstructGPT, Constitutional AI

W24 · M6
LeetCode + ML Design
DSA, system design, scratch impl.

W25-28 · M7
Specialization Track
A: Multimodal · B: Safety · C: Code

W29 · M8
Portfolio Polish
READMEs, demos, arXiv, blog

W30-31 · M8
Mock Interviews + OSS
interviewing.io, contributions

W32-36 · M9
Applications + Interviews
Apply, network, negotiate

Capstone ✦ · New in v3 ★ · Specialization

4 CAPSTONES — WHAT YOU SHIP

✦ Capstone #1 (W20)
Production RAG API — LiteLLM + Langfuse + Redis + Guardrails + security + k8s + CI evals. Load tested to 100 concurrent users.

✦ Capstone #2 (W12)
Fine-tuned LLM pipeline — synthetic data → QLoRA → DPO/SimPO → vLLM → RAGAS → Langfuse. End-to-end with Loom demo.

✦ Capstone #3 (W16)
Multi-agent Research System — arXiv agent + custom MCP server + persistent memory + LangSmith traces. Works as Claude Desktop extension.

Sebastian Raschka — Complete Resource Map

Book · Newsletter · YouTube · Blog posts mapped to each roadmap week

Why Sebastian Raschka for this roadmap

Professor-turned-Lightning AI researcher. Author of the best LLM-from-scratch book. His writing style is exceptionally clear — he explains math at the right level for someone building implementations, not just reading theory. Every resource below maps directly to a specific week.

BOOK **Build a Large Language Model From Scratch** W1–W2 · Primary
Implements GPT from token embeddings to RLHF fine-tuning in pure PyTorch. Each chapter maps to a week in Month 1–2. Use this as your coding companion alongside the papers. GitHub: rasbt/LLMs-from-scratch

NEWSLETTER **Ahead of AI — Weekly Newsletter** All months · Subscribe
Best ML research newsletter. Every issue covers 3–5 new papers with clear explanations. Subscribe and read every issue — it keeps you current without having to read every paper yourself. magazine.sebastianraschka.com

YOUTUBE **Sebastian Raschka YouTube Channel** W1–W8 · Core
Step-by-step lecture videos covering transformers, fine-tuning, evaluation. Pairs directly with the book chapters. youtube.com/@SebastianRaschka

BLOG **Understanding Large Language Models (survey)** W0 · Read first
Comprehensive survey of LLM architectures, training, and alignment. Read before starting Week 1 — gives you the map before you explore the territory. magazine.sebastianraschka.com/p/understanding-large-language-models

BLOG **Finetuning LLMs with LoRA and QLoRA: Insights from Hundreds of Experiments** W7 · Essential
The most useful LoRA hyperparameter guide. Empirical findings on rank, alpha, target modules, learning rates from hundreds of real runs. Read before you start your first QLoRA experiment. magazine.sebastianraschka.com/p/practical-tips-for-finetuning-llms

BLOG **Improving LoRA: From DoRA to QLoRA and Beyond** W7 · After LoRA paper
Full comparison of PEFT methods with empirical results. Answers the LoRA vs DoRA vs QLoRA question definitively. magazine.sebastianraschka.com/p/lora-and-dora-from-scratch

BLOG **LLM Training: RLHF and Its Variants** W23 · Alongside InstructGPT
Clearest written explanation of the full RLHF pipeline — SFT, reward model, PPO — with PyTorch code. Read alongside the InstructGPT paper as a complement. magazine.sebastianraschka.com/p/llm-training-rlhf-and-its-variants

BLOG **LLM Evaluation Deep Dive** W17 · Before DeepEval
Comprehensive guide to LLM evaluation metrics, LLM-as-judge methodology, and benchmark design. Read before setting up your CI eval pipeline. magazine.sebastianraschka.com/p/llm-evaluation

BLOG **Model Merging, Mixture of Experts, and More** W3 · MoE week
Deep dive into MoE architectures, model merging techniques, and sparse activation. Pairs with Mixtral paper. magazine.sebastianraschka.com/p/research-papers-in-january-2024

COURSE **Introduction to Deep Learning (course notes)** W0 · Math check
Raschka's full university course materials — free on GitHub. Covers backprop, CNNs, RNNs before transformers. Good for filling gaps found in Week 0 self-check. github.com/rasbt/stat453-deep-learning-ss21

How to Read and Annotate ML Papers

Three-pass method · What to annotate · How to take notes

THE THREE-PASS METHOD

1. **Pass 1 — 10 minutes:** Read title, abstract, intro, conclusion, all figure captions. Decide: is this worth a full read? What problem does it solve?
2. **Pass 2 — 1 hour:** Read fully but skip dense proofs. Understand every figure. Mark equations you don't understand. Write the core idea in one sentence in the margin.
3. **Pass 3 — 2–4 hours (print-worthy papers only):** Derive every equation. Understand every proof. Implement from scratch. Find at least one flaw or open question.

⚑ For this roadmap: do Pass 3 only on the 7 Tier-1 print papers. Do Pass 2 on the remaining 30–40 papers. Do Pass 1 on papers you're skimming for breadth.

WHAT TO WRITE IN MARGINS

- ✓ — I understand this. Can re-derive.
- **?** — I don't fully understand. Come back.
- **!** — Important result. Likely interview question.
- ★ — Key insight. Worth memorizing.
- **→impl** — I should implement this.
- **cf.** — Compare with [other paper].
- Circle undefined symbols. Write definitions in margin.
- Draw alternative diagrams if figures are unclear.
- Write the "why" next to every equation — not just what.

AFTER READING — FEYNMAN TEST

> Close the paper. Explain the core idea out loud as if teaching a first-year student. If you stumble, go back. You only truly understand what you can teach.

FOR EACH PAPER — WRITE A ONE-PAGE SUMMARY

Problem	What problem does this paper solve? What was broken before?
Key idea	One sentence. What is the novel contribution?
Method	How does it work? Key equations + intuition.
Results	What did they show? Key table/figure numbers.
Limitations	What doesn't this solve? What follow-up papers address this?
My question	Something you'd ask the author if you could.

12 PAPERS — WHAT TO FOCUS ON

PAPER	FOCUS ON
Attention Is All You Need	Derive every equation in §3. Especially §3.2 attention, §3.4 embeddings.
LoRA	§4 — the BA decomposition. Why r ≪ min(d,k) works.
QLoRA	§2.1 NF4 quantization. §3 double quantization. Memory calculation in Table 1.
DPO	§4 — the full derivation from RLHF to DPO. The Bradley-Terry model connection.
InstructGPT	§3 — RM training. §3 PPO setup. Table 6 alignment tax results.
ReAct	Figure 1 — the TAO loop. Table 1 benchmark comparison.
CoT	Figure 1 — standard vs chain-of-thought. Table 2 benchmark results.
FlashAttention-2	Algorithm 1 (tiling). §3 recomputation explanation. Figure 2 memory comparison.
Toy Models of Superposition	Every figure. §3 the geometry of superposition. §5 implications.
Constitutional AI	Figure 1 — the pipeline. §3 SL-CAI. §4 RL-CAI with RLAIF.
BERT	§3.1 pre-training objectives. Figure 1 architecture.
Scaling Laws	Figure 1, 6, 7. The power-law plots. §4 allocation.

Tokenization + Embeddings + BERT

BPE · WordPiece · BERT pre-training · Word2Vec · Sentence embeddings

BPE TOKENIZER — HOW IT WORKS

1. **Initialize:** character vocabulary. Each word = sequence of chars + end-of-word token ⟨/w⟩
2. **Count pairs:** find most frequent adjacent pair in corpus (e.g. "t h" appearing 800 times)
3. **Merge:** create new token "th". Replace all "t h" with "th" in corpus.
4. **Repeat** for N merge operations (GPT-2: 50,000 merges)

⚑ BPE balances vocabulary size vs token length. Common words get single tokens. Rare words split into subwords. Unknown words handled via character fallback.

TOKENIZER COMPARISON

METHOD	USED BY	KEY DIFFERENCE
BPE	GPT-2/3/4, LLaMA	Frequency-based merges
WordPiece	BERT	Likelihood-based merges
SentencePiece	T5, LLaMA-2	Language-agnostic, no whitespace assumption
Unigram	XLNet, ALBERT	Probabilistic, can remove tokens

EMBEDDING TYPES

Token embed.	Token ID → dense vector (learned lookup table). Dim = d_model
Positional	Encodes token position. Sinusoidal (fixed) or RoPE (rotated Q,K)
Segment	BERT-specific: which sentence (A vs B) the token belongs to
Sentence embed	Whole sentence → single vector. Use [CLS] token or mean pooling

BERT PRE-TRAINING OBJECTIVES

MASKED LANGUAGE MODELING (MLM)

```
15% of tokens replaced:
80% → [MASK], 10% → random, 10% → unchanged
Predict original token from context
```

MLM forces bidirectional context — each token can attend to all others. Unlike GPT (left-to-right only).

NEXT SENTENCE PREDICTION (NSP)

```
Input: [CLS] Sentence A [SEP] Sentence B [SEP]
Predict: IsNext (50%) or NotNext (50%)
```

NSP is now considered less useful — RoBERTa removed it and improved BERT. But know it for the BERT paper.

FINE-TUNING BERT — TASK PATTERNS

TASK	INPUT	OUTPUT
Classification	[CLS] + text	[CLS] embedding → linear
Token labeling	[CLS] + text	Each token → linear
Question answering	[CLS] Q [SEP] A [SEP]	Start + end span
Sentence similarity	[CLS] S1 [SEP] S2 [SEP]	[CLS] → linear

KEY INTERVIEW QUESTIONS

Why does BERT use bidirectional attention while GPT uses causal?

BERT is designed for understanding tasks (classification, QA) where full context is available. GPT is designed for generation where future tokens aren't available at decode time. Different objectives → different attention masks.

Why replace 10% of masked tokens with random tokens in MLM?

Without random replacement, the model learns that [MASK] tokens always need prediction and ignores non-masked tokens during fine-tuning. Random tokens force the model to maintain good representations of all tokens.

Data Curation + Synthetic Data Generation

Self-Instruct · Phi-1 · MinHash dedup · ChatML format · Distilabel

DATA QUALITY HIERARCHY (LIMA + PHI-1 INSIGHT)

> 1000 expert-curated examples > 100,000 scraped examples. Quality signals: diversity, correctness, clarity, appropriate difficulty. Quantity is the last lever to pull.

CHATML FORMAT — STANDARD FINE-TUNING STRUCTURE

```
◁im_start▷system
You are a helpful assistant.◁im_end▷
◁im_start▷user
What is LoRA?◁im_end▷
◁im_start▷assistant
LoRA is...◁im_end▷
```

Used by Mistral, LLaMA-3, Qwen. Each conversation has system/user/assistant roles. im_start/im_end are special tokens added to vocabulary.

SELF-INSTRUCT PIPELINE (WANG ET AL., 2022)

1. **Seed tasks:** 175 manually written instruction-response pairs
2. **Task generation:** GPT samples 8 random seeds → generates 20 new instructions
3. **Filter:** remove exact duplicates, low-quality, non-English
4. **Response gen:** GPT generates response for each instruction
5. **Fine-tune:** use instruction-response pairs to fine-tune base model

⚑ Alpaca (Stanford) applied Self-Instruct to LLaMA using GPT-3.5 — generated 52K examples for $500. Result was competitive with InstructGPT-175B at the time.

MINHASH NEAR-DEDUPLICATION

1. **Shingling:** convert each document to a set of n-grams (n=5)
2. **MinHash:** apply k hash functions to shingle set → k-dim signature vector
3. **LSH:** bucket similar signatures together (Jaccard similarity estimation)
4. **Remove:** keep one doc per near-duplicate cluster (Jaccard > 0.8)

JACCARD SIMILARITY

```
J(A,B) = |A ∩ B| / |A ∪ B|
MinHash estimates this in O(k) instead of
O(|A|+|B|)
```

DATA PIPELINE CHECKLIST

STAGE	ACTION	TOOL
Collect	Scrape + download domain data	requests, BeautifulSoup
Clean	Remove HTML, short docs, non-target-lang	langdetect, regex
Deduplicate	Exact + near dedup (Jaccard > 0.8)	datasketch, MinHash
Quality filter	Perplexity filter, length filter	KenLM, custom
Format	Convert to ChatML	datasets, custom
Split	Train/val/test (90/5/5)	sklearn, datasets
Log stats	Token count, length dist., topic dist.	W&B, pandas

PHI-1 KEY FINDING (GUNASEKAR ET AL., 2023)

> "Textbooks are all you need" — a 1.3B model trained on 1B synthetic textbook-quality tokens matched or exceeded 12B+ models trained on 100B web-scraped tokens. The synthetic data was generated by GPT-4 with explicit "write a clear, pedagogical explanation" prompts.

Specialization Tracks — M7 Deep Dive

Track A: Multimodal · Track B: Alignment/Safety · Track C: Reasoning+Code

Track A — Multimodal

Target: Google DeepMind

PAPERS

- CLIP (Radford, 2021) arXiv:2103.00020
- LLaVA (Liu, 2023) arXiv:2304.08485
- Flamingo (Alayrac, 2022) arXiv:2204.14198
- Gemini 1.5 (2024) arXiv:2403.05530
- InternVL2 (Chen, 2024) arXiv:2404.16821

KEY CONCEPTS

`CLIP loss`	Contrastive: maximize cos(img,text) for matched pairs, minimize for unmatched
`Visual tokens`	Image patches → embeddings fed as context to LLM alongside text tokens
`Cross-modal`	LLM attends over both text tokens and image patch embeddings
`JAX/Flax`	Google's ML framework. JIT compilation, functional style, TPU-native

BUILD DELIVERABLE

- CLIP image search engine
- LLaVA multimodal QA pipeline
- PDF → OCR + layout → VLM QA
- Simple JAX model rewrite

Track B — Alignment/Safety

Target: Anthropic

PAPERS

- Constitutional AI (Bai, 2022) arXiv:2212.08073
- Sleeper Agents (Hubinger, 2024) arXiv:2401.05566
- Representation Engineering (Zou, 2023) arXiv:2310.01405
- Activation Steering (Turner, 2023) arXiv:2308.10248
- ARENA Chapter 2 — arena-uk.streamlit.app

KEY CONCEPTS

`Deceptive align.`	Model appears safe in training but activates harmful behavior on trigger (Sleeper Agents)
`Constitution`	Set of principles used to critique + revise model outputs (no human labels)
`RepE`	Linear reading vectors identify concepts in representation space — more robust than probing
`ActAdd`	Add steering vectors to residual stream to shift model behavior at inference

BUILD DELIVERABLE

- CAI critique-revise-prefer pipeline
- Activation steering experiment
- Automated red-teaming pipeline
- Replicate Sleeper Agents result

Track C — Reasoning+Code

Target: OpenAI

PAPERS

- SWE-bench (Jimenez, 2023) arXiv:2310.06770
- Let's Verify Step by Step (Lightman, 2023) arXiv:2305.20050
- Scaling Test-Time Compute (Snell, 2024) arXiv:2408.03314
- AlphaCode 2 (DeepMind, 2023)

KEY CONCEPTS

`SWE-bench`	246 real GitHub issues. Eval: does model fix the bug and pass existing tests?
`PRM`	Scores each reasoning step — catches "right answer, wrong method"
`Test-time scale`	o1/o3: more inference compute → better reasoning. PRM guides beam search.
`Best-of-N`	Generate N solutions, pick best by PRM score. Simple but effective baseline.

BUILD DELIVERABLE

- Code agent on SWE-bench-lite
- PRM for step-by-step code scoring
- Best-of-N vs PRM-beam comparison
- Benchmark on 20 SWE-bench instances

CHOOSING YOUR TRACK

IF YOU WANT TO...	PICK	WHY
`Work on Gemini, Gato, multimodal foundation models`	Track A	DeepMind's primary research direction. JAX fluency helps significantly.
`Work on Claude, interpretability, safety research`	Track B	Anthropic's core. SAE work + CAI pipeline = direct match to their research agenda.
`Work on o1/o3, Codex, reasoning models`	Track C	OpenAI's current focus. PRM + test-time scaling = exactly what they're hiring for.

Portfolio + Job Strategy

Hosting · Networking · What labs actually want to see · Salary benchmarks

HOSTING YOUR ROADMAP (TRACK PROGRESS)

OPTION	HOW	BEST FOR
GitHub Pages	Push index.html → Settings → Pages	Recommended. Free, fast, URL stays stable.
Netlify Drop	Drag index.html to drop.netlify.app	Instant — no account needed.
Vercel	Connect GitHub repo → auto-deploys	Best if you update the file often.

⚑ localStorage is device-specific. Use the Export Progress button in the app to get a URL encoding your progress — open that URL on any device to restore.

WHAT RESEARCH LABS ACTUALLY HIRE FOR

Anthropic	Interpretability experiments, safety research, strong Python + PyTorch, Constitutional AI understanding
OpenAI	RLHF/alignment, code generation, strong DSA (hard LeetCode), research papers published or in progress
DeepMind	Multimodal, JAX fluency, strong theoretical ML, research contribution (workshop paper or arXiv)

4 CAPSTONES — POSITIONING STATEMENT

Capstone #1	Proves: production engineering. For MLOps/Applied scientist roles.
Capstone #2	Proves: full training pipeline. For fine-tuning/alignment engineer roles.
Capstone #3	Proves: systems thinking. For research engineer / applied AI roles.
Specialization	Proves: depth. For research scientist / research engineer roles.

COLD OUTREACH TEMPLATE (3 SENTENCES MAX)

```
Hi [name],
I read your paper on [specific paper] and implemented [specific part] — I found [specific observation].
I'm preparing for research roles and would love to ask you one question: [specific, answerable question].
Here's my implementation: [GitHub link].
```

Never ask for a job in the first message. The goal is a reply. A reply leads to a conversation. A conversation leads to a referral.

SALARY BENCHMARKS (LEVELS.FYI — 2024)

ROLE	COMPANY	TOTAL COMP (USD)
Research Engineer L4	Anthropic	$250K–$400K
Research Scientist	Anthropic	$300K–$500K
Research Engineer L5	OpenAI	$300K–$500K
Research Scientist	OpenAI	$350K–$600K
Research Engineer	DeepMind	$200K–$350K
Staff Research Eng.	DeepMind	$300K–$450K

⚑ Indian remote roles: typically 30–40% of US comp. Pune-based startups building on LLMs: ₹15–35 LPA for strong LLM engineers with portfolio. International remote: negotiate for 60–80% of US comp.

TIMELINE CHECKPOINTS

Week 8	Fine-tuned model on HuggingFace Hub with eval scores
Week 16	Working MCP server + research agent demo video live
Week 24	ml-interview-prep repo public with 5 from-scratch implementations
Week 28	arXiv preprint submitted
Week 32	All 4 capstones live + blog + open source contributions

ALL MONTHS · SKILLS

Skills You Gain — Complete Reference

What you can do, explain, build, and derive after each month

AFTER MONTH 1 (W0–W4)

- Derive scaled dot-product attention from scratch
- Explain RoPE vs sinusoidal PE — when each is preferred
- Implement a full transformer encoder-decoder in PyTorch
- Explain Kaplan vs Chinchilla disagreement with numbers
- Build a BPE tokenizer on a custom corpus
- Explain MoE routing + load balancing
- Explain the Mamba selective scan mechanism
- Build and evaluate a hybrid RAG pipeline with RAGAS

AFTER MONTH 2 (W5–W8)

- Add prompt injection detection to any LLM pipeline
- Build a production FastAPI service with streaming + CI evals
- Derive LoRA rank decomposition — explain why $r \ll d$ works
- Implement QLoRA fine-tuning — understand NF4 quantization
- Derive DPO from the RLHF objective via Bradley-Terry
- Run SFT → DPO → SimPO training pipeline end-to-end
- Evaluate alignment with LLM-as-judge (DeepEval)

AFTER MONTH 3 (W9–W12)

- Build a synthetic data generation pipeline (Self-Instruct style)
- Explain why Phi-style data beats web scrapes at same token count
- Serve a fine-tuned model with vLLM — explain PagedAttention
- Benchmark: tokens/sec, TTFT, p50/p95/p99 latency under load
- Build a RAFT training dataset with distractor contexts
- Show measurable RAGAS improvement: base+RAG vs RAFT+RAG

AFTER MONTH 4 (W13–W16)

- Implement ReAct loop from scratch — no frameworks
- Build a LangGraph agent with identical behavior
- Implement Tree of Thoughts for multi-branch reasoning
- Add Pydantic validation — zero silent agent failures
- Evaluate agents quantitatively (AgentBench-style)
- Build + publish a custom MCP server (works in Claude Desktop)
- Build a STORM-style collaborative multi-agent system

AFTER MONTH 5 (W17–W20)

- Write DeepEval custom metrics as pytest tests
- Set up GitHub Actions CI that fails PRs on eval score drop
- Run automated red-teaming on any LLM with Garak
- Build a 4-layer security pipeline with measurable detection rates
- Write a publishable LLM security audit report
- Deploy to Kubernetes with HPA autoscaling
- Show 100 concurrent users on k6 load test

AFTER MONTH 6 (W21–W24)

- Explain FlashAttention tiling — why $O(N^2) \rightarrow O(N)$ HBM reads
- Implement multi-GPU DDP training with gradient profiling
- Explain ZeRO stages 1/2/3 and when to use each
- Train a sparse autoencoder — find interpretable features
- Run activation steering experiments — shift model behavior
- Build full RLHF + RLAIF pipeline (mirrors Anthropic/OpenAI training)
- Implement attention + backprop in under 45 minutes from memory

AFTER MONTH 7 (W25–W28)

- Track A: CLIP image search + LLaVA VQA + JAX model
- Track B: CAI pipeline + activation steering + safety eval
- Track C: Code agent on SWE-bench + PRM for reasoning
- arXiv preprint submitted on specialization project
- Can discuss your specialization area at researcher level

AFTER MONTHS 8–9 (W29–W36)

- 4 capstones with demo videos, eval results, architecture diagrams
- 4 technical blog posts published on Medium/Substack
- 2–3 merged open source contributions (HuggingFace, vLLM, LlamaIndex)
- Mock interview fluency — explain any paper in 5 minutes
- Active ML Twitter presence with engaged research community following
- Cold outreach conversations with researchers at target labs

⚑ The one-sentence version: after this roadmap, you can build, train, evaluate, secure, and explain every component of a modern LLM system — and do so at the level expected of a Research Engineer at Anthropic, OpenAI, or DeepMind.